



Tanium Connector — Credential Provisioning Guide

Version 0.1.0, 2026-07-20

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	2
Mapping values into Crogl.....	2
Verifying the setup.....	3

Overview

The Crogl Tanium connector reads data from four Tanium Cloud modules — Asset, Comply, Patch, and Threat Response — using a single **Tanium API token**, sent as the **session** header on every call. There is no OAuth flow and no app registration; one token, scoped to a read-only persona, covers both call paths this connector uses (Tanium Gateway's GraphQL API for Asset/Comply/Patch, and the legacy Threat Response REST API).

Generating that token requires **administrator access to Tanium Console** (specifically the Administration > Permissions area — a Tanium role, not a Crogl role). If you're an FDE walking a customer through this live, expect their Tanium administrator to drive the console while you confirm the resulting values.

IMPORTANT

This connector targets **Tanium Cloud only**. On-premises Tanium deployments have a different API surface per Tanium's own documentation, and this guide has not been checked against an on-prem console.

IMPORTANT

This guide's Threat Response section is **best-effort, not confirmed against a live Tanium console**. Tanium's own public developer docs for Threat Response's REST API return a 404 at the time of writing, and Tanium's release notes state Threat Response 4.0 changed its API — meaning the actual current menu path or permission names inside the Threat Response workbench's own in-product API help (Workbench > Help > API) may differ from what's described here. Everything else in this guide (Gateway API token creation, Asset/Comply/Patch permissions) is confirmed against Tanium's live, authenticated Gateway User Guide.

What kind of account to create

Tanium's model is not an app registration; it is an **API token** tied to a **persona** (a named bundle of one or more RBAC roles) rather than a dedicated service-user login. Create it from Administration > Permissions, not from ordinary user management. The token authenticates as whatever persona/role it's issued under — so the permissions granted to that persona (see below) are what actually gate this connector's access, not the token itself.

Permissions to grant

Grant the **narrowest read-only persona** that still covers every module this connector queries:

- **Gateway execute permission** — required to submit any request through Gateway at all (covers Asset, Comply, and Patch reads). Tanium's Gateway User Guide documents building a read-only persona for this by combining the Gateway API execute permission with read-only Platform Content permissions (Filter Group Read, Sensor Read).
- **Patch Show** — required for **query-patch-definitions**. This is a Patch-specific permission, not covered by the generic Gateway read-only role above.

- **Patch Deployment Read** — required for `get-patch-deployment`.
- **Asset and Comply** have **no separate named RBAC permission** in Tanium's docs beyond the module being licensed/enabled for the tenant and the generic Gateway execute permission above — if Asset or Comply queries return no data, check the module's own workbench for whether it's licensed and has scanned data yet, not the persona's permissions.
- **Threat Response (best-guess — verify against the customer's Threat Response workbench):** the legacy REST API this connector uses for alerts likely requires its own Threat Response read permission, granted the same way as any other module-specific Tanium permission (Administration > Permissions > Roles). The exact permission name is not confirmed here — check Threat Response's own workbench (Help > API) for the current requirement before assuming a specific name.

There is no separate admin-consent or approval step as in an OAuth flow; the only gate is that an administrator must have rights to create the API token and assign it to a persona with the permissions above.

Generating the credential

1. Sign in to Tanium Console as an administrator.
2. Go to **Administration > Permissions > Roles** and create (or identify) a read-only role covering the permissions in [Permissions to grant](#).
3. Go to **Administration > Permissions > Personas** and create a persona that includes that role, then assign the intended user group / computer group scope.
4. Go to **Administration > Permissions > API Tokens** (or generate the token directly from Gateway itself — Tanium's docs note both paths are supported) and create a new API token under that persona.
5. **Copy the generated token string immediately.** Treat it as shown once — if it's lost, generate a new token rather than trying to recover the value.

You also need the tenant's **Tanium Cloud API host** — the `-api` subdomain Gateway itself uses, e.g. `acme-api.cloud.tanium.com` (no scheme, no path). This is distinct from the ordinary Tanium Console URL an analyst would browse to (which typically omits the `-api` segment) — confirm the `-api` host with the customer's Tanium admin rather than assuming it's a simple prefix substitution, since Gateway's own docs only describe the pattern, not a universal derivation rule.

Mapping values into Crogl

Two values map to `community/tanium/connector.json`'s `'credential_schema.fields'` when creating the connector instance in Crogl:

Crogl field	Where it came from	Notes
<code>api_host</code>	The tenant's Tanium Cloud API host (the <code>-api</code> subdomain)	No <code>https://</code> prefix and no path — just the host (e.g. <code>acme-api.cloud.tanium.com</code>). Crogl builds the base URL <code>https://<api_host></code> for you.
<code>api_token</code>	Administration > Permissions > API Tokens > the generated token string	Treat as a password. Never paste it into chat, a ticket, or this document. Crogl sends it as the <code>session</code> header automatically once entered.

Verifying the setup

Right after creating the connector instance in Crogl, confirm the setup module by module — a token that works for Asset can still fail on Patch or Threat Response if the persona is missing that module's specific permission:

- **Success:** `tanium.py query-asset-products --limit 5` (or the equivalent natural-language ask) returns `HTTP 200` with product rows, or a genuinely empty result if the tenant has no cataloged Asset products yet. An empty result on a narrow filter is a valid, successful response, not an error.
- **GraphQL errors in a 200 response:** Gateway can report a failed query with an HTTP 200 status and a top-level `errors` array instead of a 4xx/5xx — read the embedded error text; it usually names the missing permission or the malformed field directly.
- **401/403 on any Gateway call:** the token is invalid/expired, or the persona lacks the Gateway execute permission — regenerate the token or widen the persona's role (see [Permissions to grant](#)).
- **403 specifically on `query-patch-definitions` or `get-patch-deployment` while other modules work:** the persona has generic Gateway access but is missing the Patch-specific "Patch Show" / "Patch Deployment Read" permission — these are granted separately from the generic Gateway read-only role.
- **Comply/Patch queries succeed but return no findings/applicability rows:** check the relevant workbench (Comply or Patch) directly for whether a scan/evaluation has actually run for the target computer group yet — a persona and query can both be correct while the tenant simply has no data yet.
- **Threat Response queries fail:** given how unverified this module's transport is (see the Overview's Threat Response caveat), don't assume a 403 here is necessarily a permissions problem the same way it would be for Asset/Comply/Patch — it could equally be a stale endpoint path from an API version change. Check the Threat Response workbench's own Help > API page for the tenant's actual current API documentation before concluding it's a permissions issue.